Objetivos:

- Comprender la diferencia entre el concepto de clase e instancia
- Herencia de Clases
- Introducción a Colecciones
- DOO
- UML y lenguaje de modelado.

Ejercicio 1.

Introducción Clases: Una clase es una descripción, un molde, de un conjunto de objetos que manifiestan la misma semántica, operaciones, atributos, y relaciones. Comparten comportamiento y estructura común.

I**ntroducción Instancias:** Es un ejemplo concreto de un objeto que respeta el molde de la clase: semántica, operaciones, atributos y relaciones.

a) En la siguiente lista identifica Clases y sus instancias (cada ítem en la lista se refiere a una Clase o a una instancia de alguna de las clases en la lista), por ejemplo: `Materia` es una Clase, metodologías de programación es una instancia de esta Clase. `Docente` es una Clase, Federico Naso es una instancia de esta **Clase**.

`Carrera universitaria, UNNOBA, abogacía, IPI, metodologías de programación, IO, facultad, decano, Sebastian Sotille, materia, paradigmas de programación, Docente, universidad, facultad de Derecho, Luciana Balbi, lenguaje de programación, Pascal, Smalltalk, IPOO, Redes, Guillermo Tamarit, Carlos Di Cicco, facultad de Ciencias Exactas, UNLP, licenciatura en informática, UBA, Federico Naso, C++, Lisp, Python, BBDD.`

b) En la siguiente lista identifica Clases y subClases. Arme la jerarquía correspondiente. Como, por ejemplo, un “Círculo es una Elipse”.

`Figura, Figura Abierta, Segmento, Punto, Polígono, Triángulo, Rectángulo, Cuadrado, Círculo Elipse, Hexágono, Figura Cerrada, Rombo, Pentágono.`

Ejercicio 2.

Una afamada compañía que se dedica a innovar en aplicaciones de World Wide Web está interesada en modelar álbumes de fotos de forma electrónica.

Interesados por los resultados que esperan obtener, nos piden que modelemos e implementemos álbumes de fotos. Las fotos no indican un lugar determinado en el álbum. El álbum de fotos puede usarse de la siguiente manera:

- Agregando las fotos de a una.
- Agregando una colección de fotos al álbum.
- Eliminando una foto de un álbum.
- Eliminando todas las fotos de un álbum.

Los álbumes no tienen límite. Es decir, son infinitos.

a) Implemente en JAVA-LIKE las clases y métodos que crea necesarios para poder mandar a los álbumes los mensajes para:

   i. agregar una foto,
   ii. agregar varias fotos,
   iii. ver cuántos elementos hay,
   iv. limpiar álbum,
   v. asignar un nombre al álbum,
   vi. etc.

Ejercicio 3.

Modifique el ejercicio 12 del práctico 1 para además de `vehículo` tener, `auto`, `moto`, `colectivo` y camión. De las `motos` además de la información heredada de `vehículo` se tiene: cilindrada y tipo (ej: scooter, motocross, enduro, cruiser, chopper, etc). En el caso de los `autos` se conoce la tracción (simple delantera, simple trasera y 4x4), del `colectivo` se conoce la cantidad de asientos y si es interno o de larga distancia. Del `camión` se conoce su tara y su largo.

a) Realice el diagrama en UML

b) Implemente en JAVA-Like

Ejercicio 4.

Se desea modelar un `Editor` de `Figuras`. Un editor de figuras posee una colección de figuras, las cuales pueden ser: `cuadrado`, `círculo`, `triángulo`, `líneas`. Todas ellas tienen un color y los datos necesarios para calcular el área según qué figura es. Todas entienden los mensajes:

i. void setColor(String color) -> Setea el color de la figura

ii. String getColor() -> Retorna el color de la figura

iii. double getArea() ->/ Retorna el área de la figura

iv. void pintar() -> Imprime en consola "Pintando FIGURA COLOR" por ejemplo. "Pintando: círculo rojo"

El editor entiende los siguientes mensajes:

i. void pintar(); // Que pinta todas las figuras de la colección.

ii. double calcularArea(); //que retorna la suma de las áreas de todas las figuras.

Además, entiende los mensajes para agregar y borrar figuras de su colección.

a) Realice el diagrama en UML

b) Implemente en JAVA-Like

Ejercicio 5.

Dada una `Librería` que tiene una colección de `Libros`. De cada libro se conoce su título, autor, fecha de publicación, cada personaje y cantidad de copias en stock.

a) Realice el diagrama de clases en UML

b) Implemente al menos los siguientes mensajes

i. agregarLibro(Libro unLibro) -> Agrega un libro a la colección

ii. eliminarLibro(Libro unLibro) -> Elimina un libro de la colección

iii. cantidadDeLibros() -> Retorna la cantidad de libros

iv. cantidadTotalDeCopias() -> Retorna la cantidad total de copias

Ejercicio 6.

Se desea modelar un **sistema de información sobre películas y series**. Las mismas cuentan con actores y directores. Además de personajes y en el caso de las series se tienen además los capítulos.

a) Realice el diagrama de clases en UML

b) Especifique todos los atributos de cada clase.

Ejercicio 7.

Se desea implementar `Conjuntos` en JAVA-LIKE. Los conjuntos son colecciones que pueden tener elementos repetidos.

Extracto de Wikipedia: http://es.wikipedia.org/wiki/Teoría_de_conjuntos

El concepto de conjunto es intuitivo y se podría definir como una "colección de objetos"; así, se puede hablar de un conjunto de personas, ciudades, gafas, lapiceros o del conjunto de objetos que hay en un momento dado encima de una mesa. Un conjunto está bien definido si se sabe si un determinado elemento pertenece o no al conjunto.

Sean A y B dos conjuntos:

Unión: Los elementos que pertenecen a A o a B o a ambos A y B, forman otro conjunto, llamado **Unión** de A y B, escrito $A \cup B$. Así pues, se tiene

$$A \cup B = \{x : x \in A \quad \vee \quad x \in B\}$$

Intersección: Los elementos comunes entre A y B forman un conjunto denominado **Intersección** de A y B, representado por $A \cap B$:

$$A \cap B = \{x : x \in A \quad \wedge \quad x \in B\}$$

Diferencia: Los elementos de un conjunto A que no se encuentran en otro conjunto B, forman otro conjunto llamado *diferencia de* A y B, representado por, $A - B$:

$$A - B = \{x : x \in A \quad \wedge \quad x \notin B\}$$

Diferencia simétrica: Se define la *diferencia simétrica* de dos conjuntos por:

$$A \Delta B = (A - B) \cup (B - A)$$

NOTA: Para facilitar el trabajo vamos a trabajar con conjuntos de `Personas`. Esto significa que nuestro conjunto solo podrá contener personas.

Se pide que:

a) Realice el diagrama en UML

b) Implemente en JAVA-LIKE la Clase `Conjuntos`

c) Los Conjuntos deben ser capaces de entender los siguientes mensajes:

   i. add(Persona p)

   ii. remove(Persona p)

   iii. unión(Set aSet)

   iv. intersección(Set aSet)

   v. diferencia(Set aSet)

   vi. diferenciaSimetrica(Set aSet)

d) Supongamos que tenemos los siguientes conjuntos:

$$A = \{2, 4, 6\} \; ; \; B = \{4, 6, 8, 10\} \; ; \; C = \{10, 14, 16, 26\}$$

Muestre ejemplo de cómo sería:

a. La unión de A y B
b. La unión de A y C.
c. La intersección de A y B.
d. La intersección de A y C.
e. La diferencia de A y B.
f. La diferencia simétrica de A y B.

Ejercicio 8.

Especificar e implementar las clases `Persona`, `Empleado`, `Administrativo` y `Técnico`. Los datos relevantes de una persona son: nombre, DNI, dirección, año de nacimiento y sexo. Un empleado es una persona que desempeña una función en alguna empresa y recibe un salario por ello.

Un técnico es un empleado que posee un título y trabaja por contrato. Es decir, cobra siempre el mismo monto que establece el contrato (su salario se especifica en el contrato y no cambia).

Los empleados administrativos reciben su salario de acuerdo a la categoría (valores numéricos 1, 2, 3 y 4) y antigüedad que posean (en años) dividida 4.

*salario = (basico * categoría) + ((básico * antigüedad) / 4)*

La clase `Empresa` tiene una variable de instancia llamada empleados que es una colección que contiene todos los empleados de la empresa.

Implementar el método *montoTotal* en la clase `Empresa`, que retorna el monto total que la empresa debe pagar, en concepto de sueldos, a sus empleados.

Ejercicio 9.



Se desea modelar un Navegador de Internet o más comúnmente llamado **Browser**, similar al Google Chrome, Edge, FireFox, etc. El Browser es sencillo, solamente puede navegar por una `Página` web por vez. Esa sería la página actual o `Current Page`, la cual conoce.
Al `Browser`**,** se le puede indicar cuál es la *current page*.
La página cuenta con una `dirección web`, una fecha de visita, el tamaño (peso de la misma) y el código HTML (solamente texto).
El browser, posee un `Historial` de páginas visitadas, las cual guarda por si el usuario desea volver a visitar. El historial tiene la funcionalidad de:

i. Agregar páginas.

ii. Eliminar páginas.

iii. Conocer la última página visitada.

iv. Retornar la cantidad de páginas visitadas

Además del historial, también se cuenta con los favoritos que son una colección de **Direcciones Webs** que el usuario guarda como direcciones preferidas.

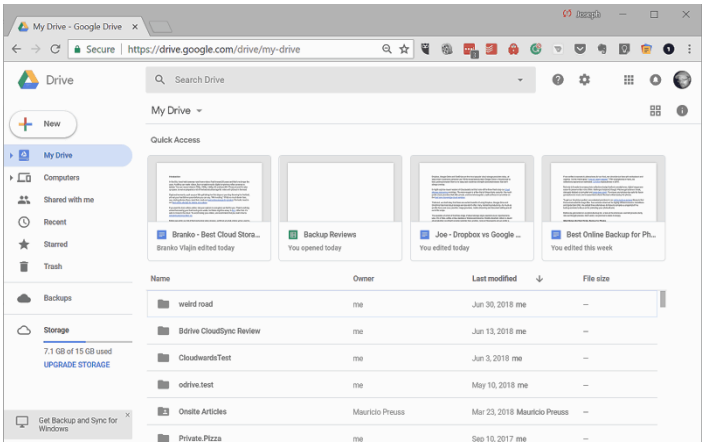
a) Diseñe en UML una solución al enunciado, identifique las clases involucradas, atributos, responsabilidades y relaciones.

Ejercicio 10.

Dadas las siguientes "Clases" arme el diagrama de clases correspondiente teniendo en cuenta herencia y composición (se debe indicar clases abstractas y cardinalidad)

`Motor, Barco, BarcoMilitar, Radar, Ancla, Capitán, Marinero, BarcoComercial, Persona, BarcoDeCarga, BarcoDePasajeros, Camarote, Cañón, Pasajero, PortaAviones, Destructor.`

Ejercicio 11.



Se desea modelar un sistema de gestión de documentos online, similar a Google Drive. El sistema está compuesto por muchos documentos.
De momento solo maneja un solo tipo de documentos.
Del sistema se conoce el dueño (una persona), el idioma y la fecha de creación del mismo.
De cada documento se conoce el nombre, el texto, la fecha de creación, el dueño y una colección de editores (estos son personas que pueden editar el documento). Los documentos se pueden compartir entre varios editores.

El sistema de gestión de documentos tiene la siguiente funcionalidad:

i. Cantidad de documentos -> retorna la cantidad total de documentos

ii. Cantidad de editores(Documento unDocumento) -> retorna la cantidad total de editores del documento.

iii. Cantidad total de editores - > retorna la cantidad total de editores de todos los documentos.

iv. agregarDocumento(Documento unDocumento) -> Agrega un documento a la colección de documentos.

v. agregarEditor(Documento unDocumento, Editor unEditor) -> Agrega un editor al documento.

a) Diseñe en UML una solución al enunciado, identifique las clases involucradas, atributos, responsabilidades y relaciones.